

Interrupts and Exceptions

COMS10015

Dr Tom Deakin

University of Bristol

Week 20

Communication

- ▶ Processors need to communicate with other devices.
- ▶ Recall the lecture on Buses.
- ▶ Need a mechanism for pausing the running program on a signal from another device.
- ▶ When this signal happens, need to transfer to running a specific routine.
- ▶ That routine then consumes the data message from the device.
- ▶ Then return to running main program when finished.

Interrupts

Definition: Interrupt

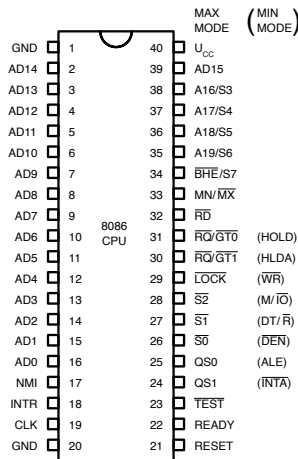
An *interrupt* is an **asynchronous** signal sent to the processor from external hardware.

- ▶ Example: keyboard input.
- ▶ The signal can arrive at any time with respect to the instruction cycle.
- ▶ This is different to exceptions which happen synchronously with instruction execution.
- ▶ Need additional architectural support for interrupts.

Detecting interrupts

- ▶ To support interrupts, processor adds an input (wire) that external devices use to send signal.
- ▶ E.g. Intel 8086 processor has two input wires/pins for interrupts: INTR and NMI.
- ▶ Processor checks for interrupts as part of instruction cycle.
- ▶ But when they might occur during the cycle is unknown, as the signal is sent from outside the processor.
- ▶ Processor normally uses a latch (flip-flop) to 'keep' the input signal (i.e., a key press is a temporary signal).
- ▶ Processors often support a few interrupt signals, or interact with an external device which helps prioritise between multiple signals.

Picture from https://en.wikipedia.org/wiki/Intel_8086#/media/File:Intel_8086_pinout.svg



Exceptions

Definition: Exception

An *exception* is a synchronous event generated by the processor when a condition occurs when executing an instruction.

In general, these conditions imply something bad has happened:

- ▶ Divide by zero.
- ▶ Unknown instruction.
- ▶ ALU overflow.
- ▶ Floating-point maths errors.
- ▶ Memory access errors.

Typically dealt with in the same way as interrupts, but the source is different.

Handling interrupts

- ▶ Once an interrupt signal has been picked up by the processor (during the execution cycle) it must deal with it.
- ▶ Typically must deal with it straight away (as soon as possible).
- ▶ This is often after the current instruction has finished executing safely, e.g., in instruction fetch part of execution cycle.
- ▶ Control is transferred to the *interrupt handler* instead of fetching next instruction.
- ▶ Similar behaviour to subroutine:
 - ▶ Program counter saved on the stack.
 - ▶ Other processor state (registers) saved to the stack.
 - ▶ Program counter updated to the address of the handler in memory
 - ▶ Program counter and registers restored once interrupt handler finished.
- ▶ Unlike subroutines, processor needs built in support for this as can happen at any time.

Architectural support for interrupts

- ▶ Processor includes an extra register to save the program counter (e.g. *spc*).
- ▶ When interrupt is detected, the *pc* is transferred to the *spc*.
- ▶ The *pc* is then set to a pre-defined memory address containing the instructions for the interrupt handler.
 - ▶ Those pre-defined addresses are called *interrupt vectors*.
 - ▶ Depending on the type interrupt use a different interrupt vector.
- ▶ At the end of the handler, the *pc* is restored to the value in *spc*.
- ▶ A status bit in the processor is also set to show that an interrupt is in progress.
 - ▶ Prevents that interrupt being interrupted, which could lead to inconsistent data.
 - ▶ Ensures mutual exclusion to the data during the interrupt.
 - ▶ Interrupt handler must clear/reset interrupt status.

Armv7-A IRQs

As a concrete example, let's look at Armv7-A IRQ exception:

1. IRQ interrupt raised by external hardware.
2. *PC* register copied to *LR* register (This is R14, but now called *LR_IRQ*).
3. *CSPR* register Mode bit set to *IRQ* mode and mask further IRQ interrupts, and copy to *SPSR*
4. *PC* set to the *IRQ* entry in the interrupt vector table, usually found at 0xFFFF0018.
5. The instruction at that address is then fetched and executed: a branch/jump instruction to the interrupt handlers.
6. The interrupt handler will push to the stack any registers corrupted by the handler.
7. The interrupt handler routine runs, finding the correct routine to run depending on the interrupt type.
8. When done; pop the variables from the stack back to registers.
9. Copy *SPSR* to *CSPR*.
10. Restore *PC* from *LR*.

Some x86 Interrupt Vectors

Vector	Description
0	Divide Error
3	Breakpoint
4	Overflow detection with INT0 instruction
6	Invalid opcode
14	Page Fault
32–255	User Defined Interrupts: external, INT instruction

Operating system

- ▶ The operating system is the lowest level of software running on a processor.
- ▶ Layer of abstraction between applications we write and the hardware.
- ▶ Interface between software and Instruction Set Architecture:
 - ▶ Sets up the processor and memory.
 - ▶ Gives access to peripheral devices (Disks, GPUs, Network, ...).
 - ▶ Provides protection and security.
 - ▶ Manages memory, data and files.
- ▶ The *kernel* is the heart of the operating system providing this functionality.
 - ▶ Controls which applications run at what time: scheduling.
 - ▶ Controls Input/Output.
- ▶ Kernel instructions loaded into memory.

Back to interrupts

- ▶ The kernel itself is a program loaded into memory.
- ▶ The interrupt vectors are part of this kernel.
- ▶ Exact code will be dependent on the type of device sending the signal
 - ▶ Keyboard entry will be different to receiving data from disk, etc.

Timing interrupts

- ▶ Processor often includes a timing unit that generates interrupts at regular interval.
- ▶ Can use this interrupt to switch to kernel periodically.
- ▶ One mechanism for running programs *concurrently* on a shared processor.
- ▶ A scheduler will share time on the processor between different programs.
- ▶ Processor is only running one at a time, switching them in and out regularly.

Further Reading

- ▶ <https://www.electronicshub.org/types-of-interrupts-and-how-to-handle-interrupts/>
- ▶ [https://en.wikipedia.org/wiki/Kernel_\(operating_system\)](https://en.wikipedia.org/wiki/Kernel_(operating_system))
- ▶ <https://en.wikipedia.org/wiki/Interrupt>
- ▶ <http://people.cs.bris.ac.uk/~dave/hexkernel.pdf>
- ▶ <http://ece425web.groups.et.byu.net/stable/labs/InterruptMechanism.html>
- ▶ [https://en.wikipedia.org/wiki/Interrupt_request_\(PC_architecture\)](https://en.wikipedia.org/wiki/Interrupt_request_(PC_architecture))
- ▶ https://en.wikibooks.org/wiki/X86_Assembly/X86_Interrupts
- ▶ <https://linux-kernel-labs.github.io/refs/heads/master/lectures/interrupts.html>